

Week 3 Progress Report

Design of Lightweight SIEM Architecture and Unified Event Schema

Introduction

This week focused on transitioning from theoretical research to system design. Building on the literature review completed in Week 2, the objective for Week 3 was to design the high-level architecture of the proposed lightweight Security Information and Event Management (SIEM) system and define a unified event schema to support log normalization and detection.

The system is designed to process pre-existing log datasets containing both security logs and audit logs. The architecture emphasizes modularity, explainability, and realistic scope for a single-node prototype.

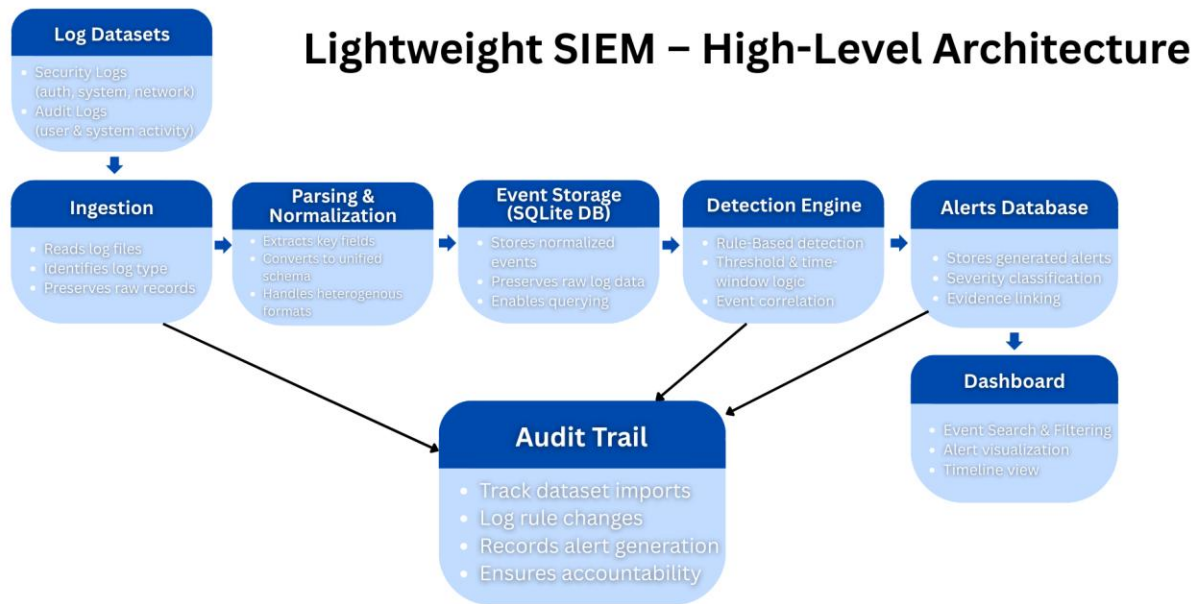
High-Level SIEM Architecture

The proposed SIEM follows a modular pipeline design. Log datasets are ingested from a pre-existing dataset, normalized into a unified event structure, stored for querying, analyzed by a detection engine, and converted into alerts for visualization. Additionally, the system maintains its own internal audit trail to ensure accountability and traceability.

The architecture consists of the following core components:

- Log Datasets
- Ingestion Module
- Parsing & Normalization Engine
- Event Storage (SQLite Database)
- Detection Engine
- Alerts Database
- Dashboard Interface
- SIEM Internal Audit Trail

The pipeline ensures separation of concerns between ingestion, normalization, storage, detection, and presentation.



Architectural Component Explanation

Log Datasets

The system processes pre-existing security and audit log files. Security logs may include authentication logs, system logs, and network-related logs. Audit logs capture user and system activity such as command execution and privilege changes.

Ingestion Module

The ingestion module reads log files, identifies the log type, and preserves the raw records to maintain traceability. This module acts as the entry point for all log data.

Parsing & Normalization Engine

The normalization engine extracts key fields such as timestamps, usernames, IP addresses, event types, and outcomes. All heterogeneous log formats are converted into a unified event schema to allow consistent querying and detection.

Event Storage (SQLite Database)

Normalized events are stored in a SQLite database. This supports structured querying, time-window analysis, and correlation. Raw log records are also preserved to maintain forensic integrity.

Detection Engine

The detection engine applies rule-based logic, threshold checks, and event correlation to

identify suspicious behaviors. Detection is performed after events are stored to enable historical analysis and multi-event correlation.

Alerts Database

When detection rules are triggered, alerts are generated and stored separately from raw events. Alerts include severity classification and evidence linking to relevant event IDs.

Dashboard Interface

The dashboard allows users to search and filter events, visualize alerts, and view event timelines.

SIEM Internal Audit Trail

The SIEM maintains its own internal audit trail. This records dataset imports, rule modifications, and alert generation. This ensures accountability and supports evaluation of system behavior.

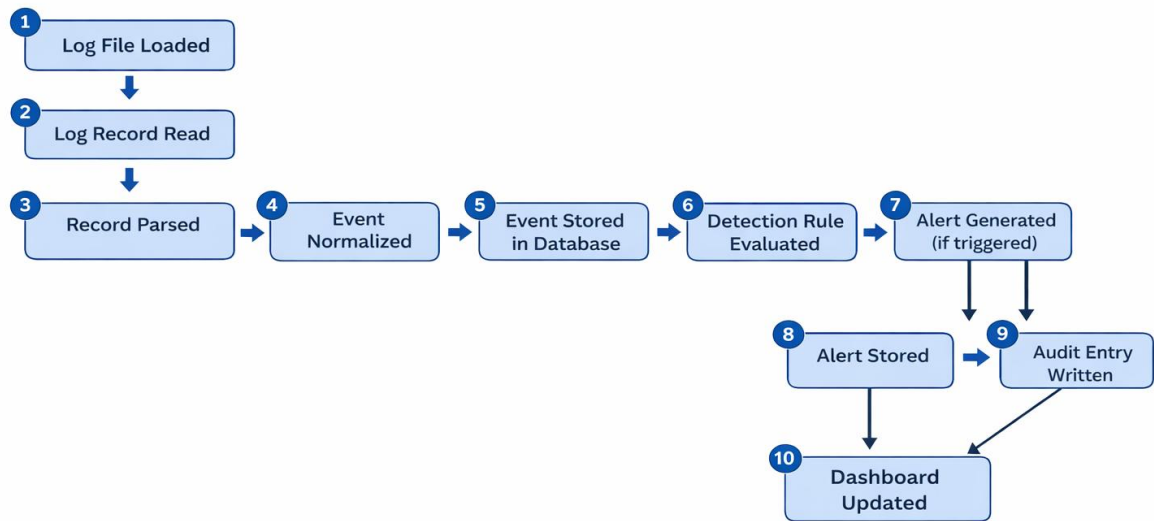
Data Flow Design

In addition to the architectural diagram, a data flow diagram was developed to illustrate the lifecycle of a single log record as it moves through the system.

The event lifecycle follows these steps:

1. Log file is selected and loaded.
2. Log record is read by the ingestion module.
3. Record is parsed and key fields are extracted.
4. Event is normalized into the unified schema.
5. Normalized event is stored in the database.
6. Detection rules are evaluated.
7. If triggered, an alert is generated.
8. Alert is stored in the alerts database.
9. An internal audit entry is recorded.
10. Dashboard is updated to reflect new events and alerts.

Data Flow in the Lightweight SIEM



Unified Event Schema

To support consistent analysis across heterogeneous log sources, a unified event schema was defined. This schema standardizes key fields required for detection and correlation.

Core Fields:

- event_id
- timestamp
- source
- event_type
- host
- user
- outcome
- src_ip
- raw_log

Optional Context Fields:

- process_name
- command
- file_path
- severity
- tags

This schema allows different log types (authentication logs, system logs, audit logs) to be normalized into a consistent structure while preserving raw data for traceability.

Design Rationale

The architecture is intentionally lightweight and modular. SQLite was selected as the storage backend due to its simplicity and suitability for a single-node prototype. Detection is performed after event storage to enable correlation and time-window analysis.

Separating events and alerts ensures clarity between raw activity and detected incidents. The inclusion of an internal audit trail strengthens accountability and supports evaluation of detection logic during later project phases.

Next Steps (Week 4)

The next phase of the project will focus on implementing the core data processing components defined in the Week 3 architecture. The primary objective for Week 4 is to operationalize the ingestion and normalization layers of the SIEM pipeline.